

An End-to-End AI Agent for Kaggle Competitions

An end-to-end LLM agent benchmarked against two frozen yardsticks on 20 Kaggle competitions

Author: Wei-Hao Huang · **School:** National Chengchi University, Department of Economics
PI: Dr. Tso-Jung Yen, Institute of Statistical Science, Academia Sinica · Summer Internship

1 · Motivation

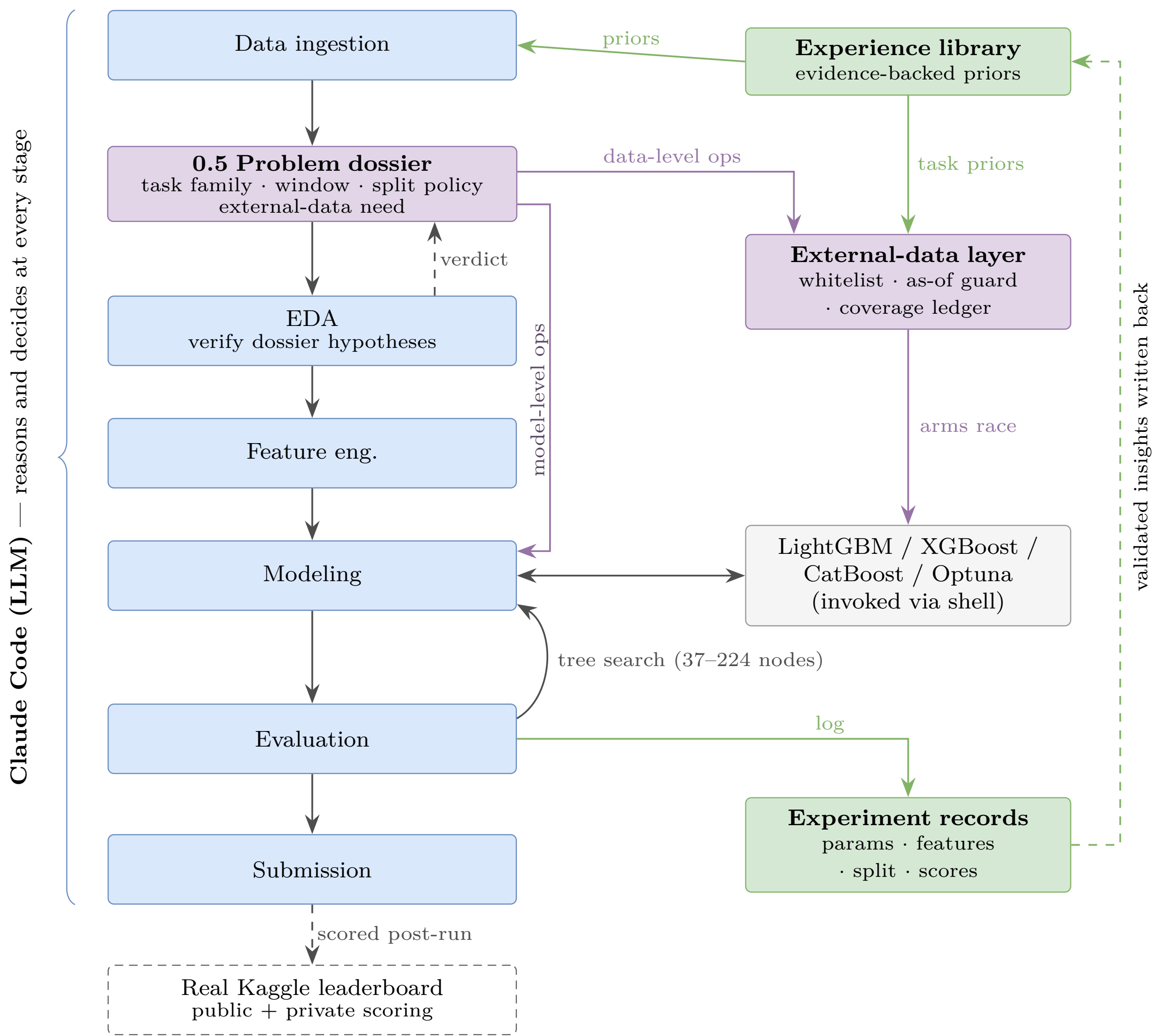
Why these tasks are hard

- **Kaggle competitions** demand the full expert loop — validation design, feature engineering, model selection, ensembling — where the difference between top entries sits at the **4th–5th decimal** of the metric.
- With gaps that small, **local cross-validation is an unreliable compass**: a pipeline’s own validation score routinely disagrees with the real leaderboard, so no local number can be trusted to pick a winner.
- **Data visualization** (EDA, diagnostics, reporting) is essential but repetitive: every dataset needs distribution, drift, and correlation views before modeling can be trusted.

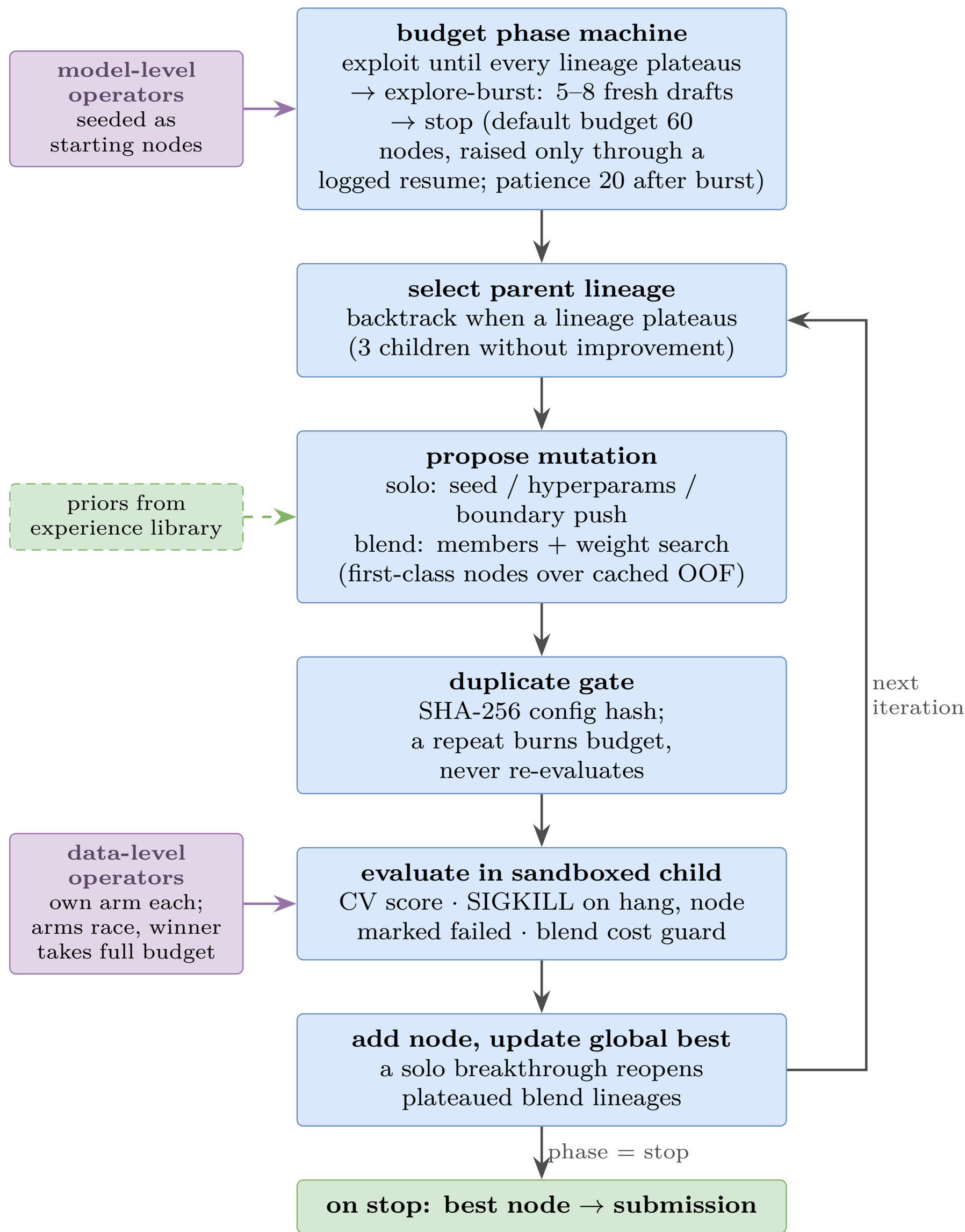
Why an AI agent

- Tireless, systematic search: 37–224 candidate configurations per competition, every experiment logged and reproducible.
- Runs the same disciplined protocol on 20 competitions serially — a human cannot hold this consistency.
- Records a per-dataset EDA pass and writes a per-competition report automatically, so every claim is grounded in an artifact.

2 · Workflow



The pipeline, read top to bottom: the LLM reasons at every stage; the **problem dossier** classifies the task and injects external data *upstream*, before any modeling; submissions are scored post-run on the real leaderboard. Purple = injection layer · green = evidence loop · gray = Auto-ML tools; dashed = feedback or post-run.



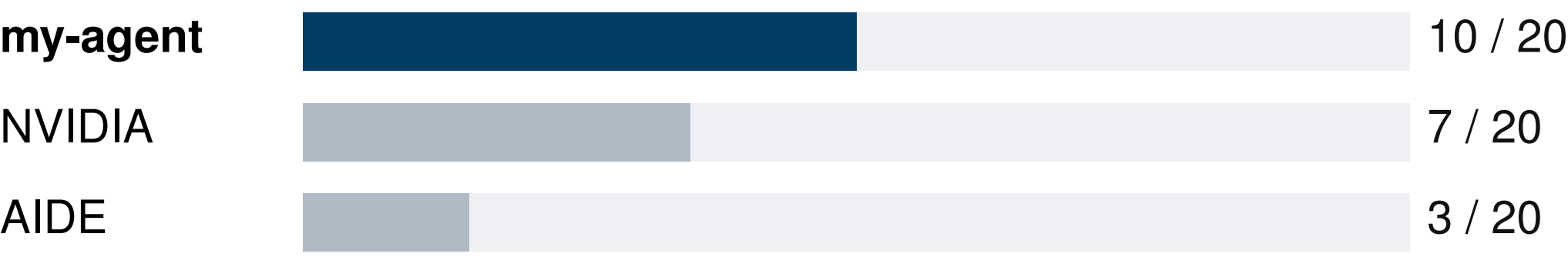
Tree-search loop: ensembles are first-class nodes; exploit until a lineage plateaus, then one forced exploration burst; the loop ends at the best-node submission.

3 · Results

Benchmark standing

All 20 competitions ran in sandboxes with a per-lane transcript audit, and all 20 are scored on the real leaderboard — as were the two yardsticks. Each of the 60 head-to-head duels was judged against the score noise measured on Kaggle’s own public/private split: 38 decided, 22 too close to call. **my-agent leads the standing — 16 wins / 8 losses (66.7% of its decided duels)** against 42.3% for each yardstick, and beats its own development-lane score on 13 of 20.

Best private score — head-to-head count



Private-leaderboard scores

Comp.	Metric	mine	NV	AIDE
s3e1	RMSE ↓	0.55275	0.55550	0.55552
s3e3	AUC ↑	0.87157	0.89537	0.86863
s3e5	QWK ↑	0.57223	0.56974	0.58138
s3e7	AUC ↑	0.90441	0.90458	0.90109
s3e9	RMSE ↓	12.3319	12.2291	12.3067
s3e11	RMSLE ↓	0.29368	0.29624	0.29445
s3e14	MAE ↓	333.186	330.716	332.316
s3e16	MAE ↓	1.33822	1.34001	1.34224
s3e19	SMAPE ↓	49.4088	48.2656	48.3413
s4e11	Acc. ↑	0.94107	0.94076	0.94056

Comp.	Metric	mine	NV	AIDE
s5e1	MAPE ↓	0.08401	0.14594	0.17917
s5e10	RMSE ↓	0.05576	0.05588	0.05579
s6e1	Error ↓	8.71516	8.73239	8.72191
afsis	MCRMSE ↓	0.48917	0.69855	0.49861
cat-in-dat	AUC ↑	0.80243	0.77084	0.80217
conway	MAE ↓	0.10414	0.11189	0.11065
s6e2	AUC ↑	0.95505	0.95508	0.95516
tps-aug	AUC ↑	0.58930	0.58730	0.59106
tps-jan	SMAPE ↓	6.29895	4.63651	10.6664
tps-sep	SMAPE ↓	28.4961	5.48351	9.72095

Private-leaderboard score of each submission; **bold** = best of the three.

Beyond tabular — three mid-complexity competitions

Competition	Metric	my-agent	NVIDIA	AIDE
us-patent (NLP)	Pearson r ↑	0.86658	0.86160	0.65821
ventilator (time series)	MAE ↓	0.16525	0.40093	0.34591
SIIM-ISIC (imaging)	AUC ↑	0.93529	0.89822	0.88579

4 · Lessons from building this AI agent

- **Put knowledge injection upstream.** At the blend stage it measured near-nothing and was abandoned; moved into problem identification it flags exactly the competitions that need external data, and **turns the canonical failure case around**.
- **Local CV is the only feedback while you iterate.** A competition returns no score until submission, unlike a task that can be graded on the spot, so an error in problem-level judgment leaves CV looking healthy while the leaderboard says otherwise.
- **Version the agent, not just the code.** An architecture change invalidates every competition that ran before it: rebuild the run root, re-hash it, and start the benchmark over rather than mixing versions in one table.
- **Isolate mechanically, then audit the transcript.** Sandboxes stop leaks you anticipated; reading what the agent actually did catches the ones you didn’t — and it catches the auditor too: ours exempted every command starting with `uv`, so the one command it existed to catch was never examined.

5 · Where AI-heavy work still falls short

- **Engineering judgment is the gap, not technique.** The code is fast and accurate, but nothing in the loop can tell at design time whether an architecture will hold. The flaws surface only once it runs — and by then a weak structure has started accumulating debt that grows with every competition added to it.
- **A pipeline will log success for work nothing performed.** Our dossier emitted post-processing operators under names the evaluator had never implemented: the instruction went out, nothing ran, no warning was raised, and the ledger recorded it as handled. The numbers still looked right — because the evaluator happened to hard-code something similar.
- **Documents can agree with each other and all be wrong.** A claim about the search budget was copied into report, poster and slides, and read as verified precisely because it was consistent everywhere. Only the source code disagreed.
- **The question gets lost in the detail.** Heavy AI use pulls a write-up toward the technical detail it can report exhaustively, and away from the one thing the reader came for.
- **AI writing habits survive review.** Redundant heading suffixes and invented terminology come back in every draft; keeping them out is manual work, not a setting.